USING THE CEDAR POLICY ENGINE

Cedar is a policy evaluation engine—it does not make access control decisions independently. Instead, it is part of a larger system that provides the data and context necessary to evaluate each request.

To use Cedar effectively, a developer needs to:

- Construct the access request, including the principal, action, resource, and any relevant contextual data.
- Provide the relevant policies for the application.
- Provide entity data representing the principals and resources involved, including any relationships or attributes that policies depend on, such as ownership or team membership.

The Cedar engine evaluates policies using the request and entity data and then issues a decision: allow, deny, or no decision.

Although Cedar itself doesn't include a built-in entity store or request formatter, several open-source tools can assist. One notable example is the Cedar Local Agent (https://github.com/cedar-policy/cedar-local-agent), a reference implementation that wraps the Cedar engine in a REST API and manages policies and entities in memory. It's an excellent way to explore and prototype Cedar-based systems without having to build all the integration yourself.

In production, most teams embed Cedar directly into their application or authorization service, integrating it with identity providers, data stores, and contextual signals. The ability to bring your own data model and evaluate policies against it is one of Cedar's main strengths.

In this example, the condition `resource.owner == principal` depends on data from ACME's internal relationship graph, not the request. Remember that the Customer Collaboration platform uses a graph to model ownership and other relationships. An edge labeled `owner` points from the `Document::"doc456"` entity to the `Employee::"bob"` entity, indicating that